Ex.No.4	**IMPLEMENTATIONOF RSA**

AIM:

To simulate the working of RSA in Virtual lab environment and to implement the same in Java/Python

THEORY:

1. One-way function

A **one-way function** is a mathematical function that is easy to compute in one direction but hard to reverse without special information.

- In RSA, the **modular exponentiation with large primes** is the one-way function.
- Given m, computing c=m^e mod n is easy.
- But given c, finding m without the private key d (i.e., solving m=c^d mod n) is computationally hard.
- This hardness relies on the difficulty of factoring large integers (the product of two large primes).

2. Key generation

RSA key generation steps:

1. Choose two large **distinct prime numbers** ppp and qqq.
2. Compute n=p×q. The number n is called the **modulus**.
3. Compute Euler's totient function φ(n)=(p−1)(q−1).
4. Choose a public exponent e such that 1<e<φ(n) and gcd(e,φ(n))=1 (usually e=65537 is used).
5. Compute the private exponent d as the modular inverse of e modulo φ(n), i.e., d≡e^−1mod φ(n).

- The **public key** is (e, n)
- The **private key** is (d, n).

3. RSA Encryption

To encrypt a message m (converted to an integer $0 \leq m < n$) using the public key (e, n):

$c = m^e \bmod n$

- c is the ciphertext.
- This operation is easy to compute using modular exponentiation.

4. RSA Decryption

To decrypt ciphertext ccc using the private key (d, n):

$m = c^d \bmod n$

- This recovers the original message m.
- Only someone with the private key d can perform this operation efficiently.

ALGORITHM:

1. Key Generation

1. Choose two large distinct prime numbers: p and q.
2. Compute $n = p \times q$.
3. Compute Euler's totient: $\varphi(n)=(p-1)\times(q-1)$.
4. Choose an integer e such that $1 < e < \varphi(n)$ and $\gcd(e,\varphi(n)) = 1$.
5. Compute the modular inverse d of e modulo $\varphi(n)$, such that: $d \times e \equiv 1 \bmod \varphi(n)$
6. Public key = (e, n), Private key = (d, n).

2. RSA Encryption

- Given the public key (e, n) and plaintext message m (as an integer with 0 ≤ m <n), compute ciphertext:

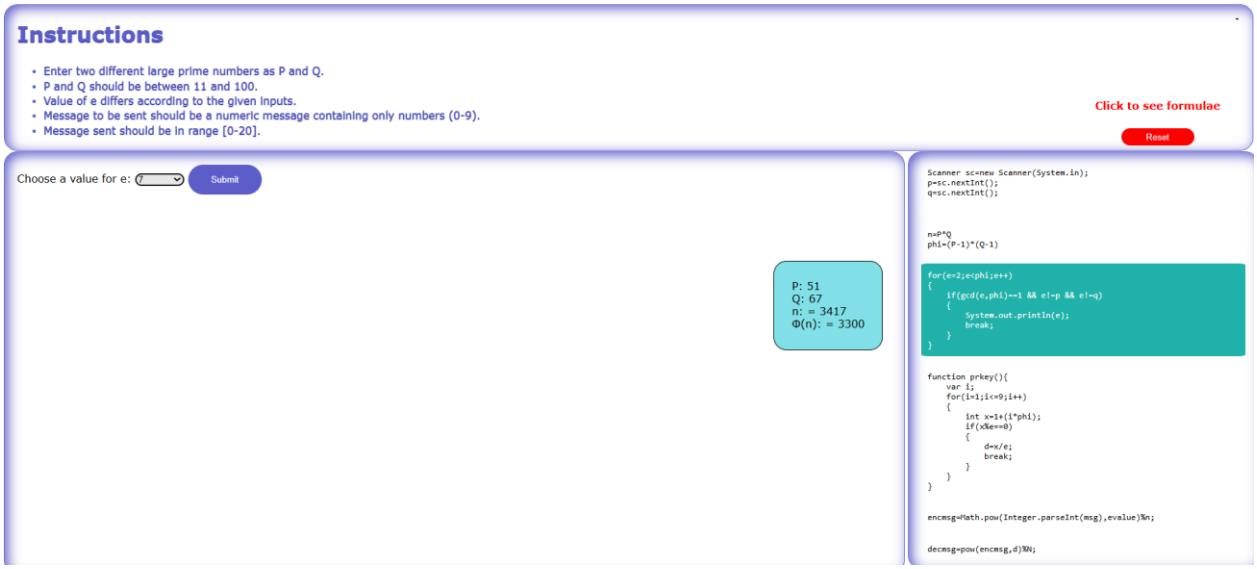
$$c = m^e \bmod n$$

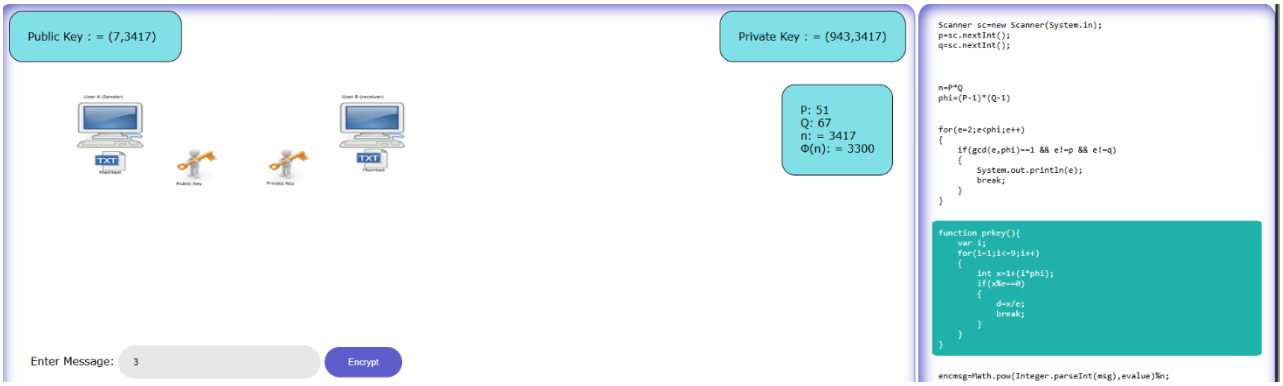
3. RSA Decryption

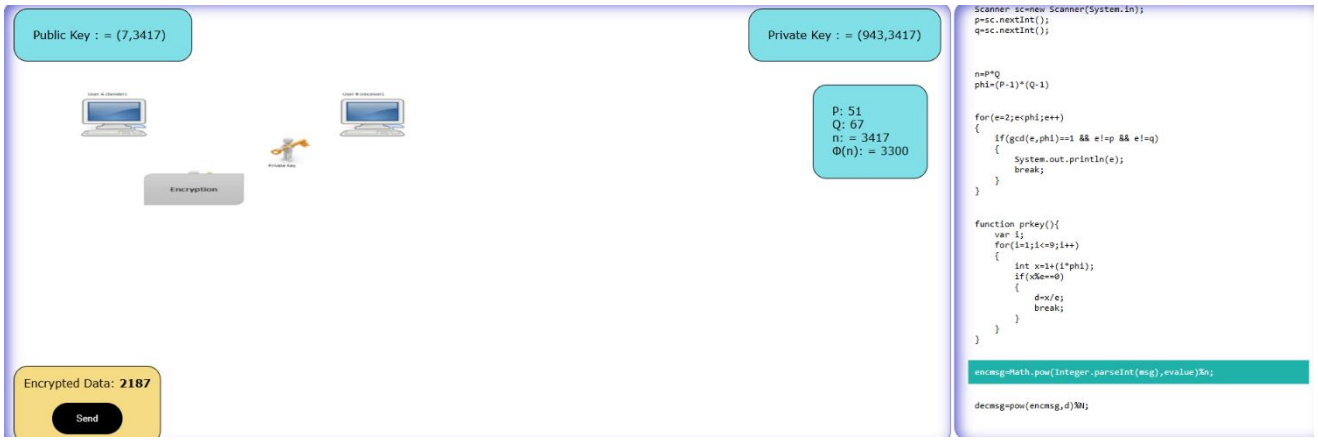
- Given the private key (d, n) and ciphertext c, recover the original message: m=

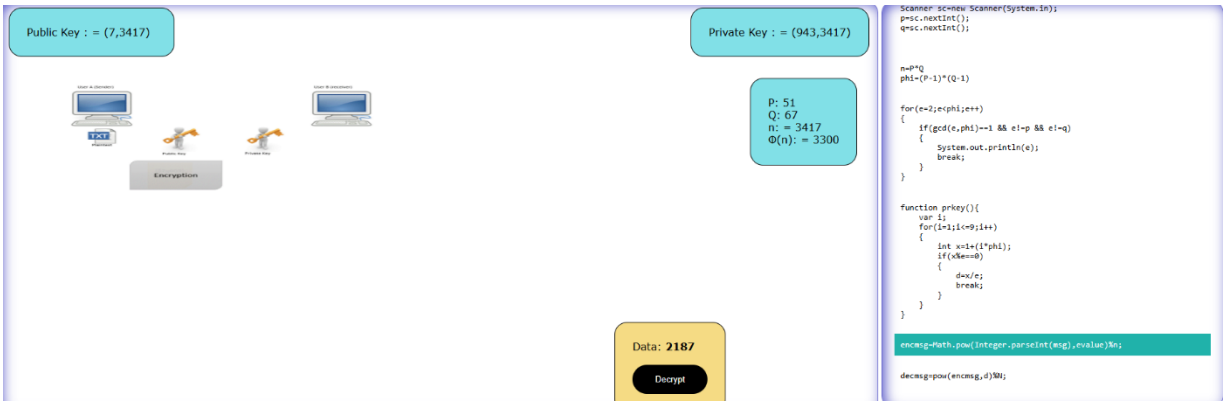
$$c^d \bmod n$$

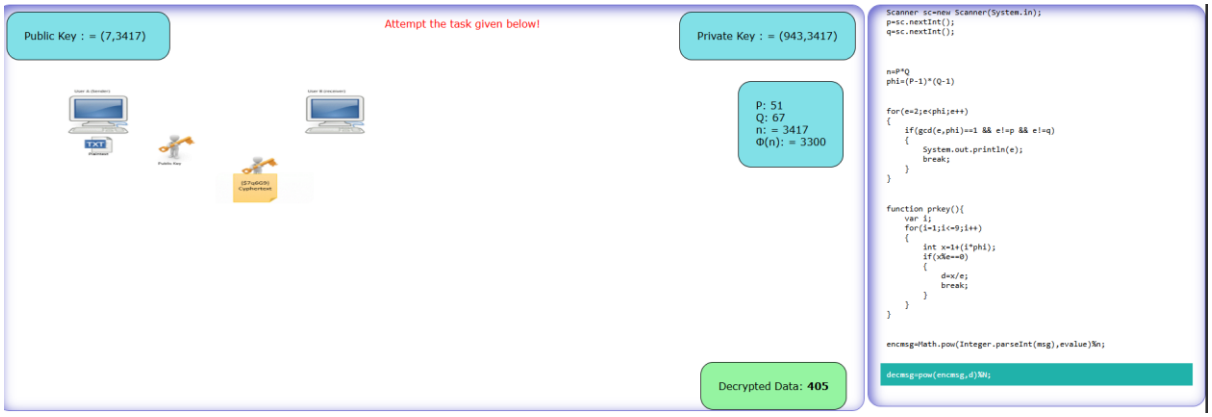
ScreenShots of simulation in Virtual labs


Instructions
Enter two different large prime numbers as P and Q.
P and Q should be between 11 and 100.
Value of e differs according to the given inputs.
Message to be sent should be a numeric message containing only numbers (0-9).
Message sent should be in range [0-20].
Click to see formulae
Reset
Choose a value for e:
Submit
P: 51
Q: 67
n: = 3417
Φ(n): = 3300
Scanner sc=new Scanner(System.in);
p=sc.nextInt();
q=sc.nextInt();
n=P*Q
phi=(P-1)*(Q-1)
for(e=2;e<phi;e++)
{
if(gcd(e,phi)==1 && e!=p && e!=q)
{
System.out.println(e);
break;
}
}
function prkey(){
var i;
for(i=1;i<=9;i++)
{
int x=1+(i*phi);
if(x%e==0)
{
d=x/e;
break;
}
}
}
encmsg=Math.pow(Integer.parseInt(msg),evalue)%n;
decmsg=pow(encmsg,d)%N;


Public Key : = (7,3417)
Private Key : = (943,3417)
P: 51
Q: 67
n: = 3417
Φ(n): = 3300
Enter Message:
3
Encrypt
Scanner sc=new Scanner(System.in);
p=sc.nextInt();
q=sc.nextInt();
n=P*Q
phi=(P-1)*(Q-1)
for(e=2;e<phi;e++)
{
if(gcd(e,phi)==1 && e!=p && e!=q)
{
System.out.println(e);
break;
}
}
function prkey(){
var i;
for(i=1;i<=9;i++)
{
int x=1+(i*phi);
if(x%e==0)
{
d=x/e;
break;
}
}
}
encmsg=Math.pow(Integer.parseInt(msg),evalue)%n;


Public Key : = (7,3417)
Private Key : = (943,3417)
P: 51
Q: 67
n: = 3417
Φ(n): = 3300
Encryption
Encrypted Data: 2187
Send
Scanner sc=new Scanner(System.in);
p=sc.nextInt();
q=sc.nextInt();
n=P*Q
phi=(P-1)*(Q-1)
for(e=2;e<phi;e++)
{
if(gcd(e,phi)==1 && e!=p && e!=q)
{
System.out.println(e);
break;
}
}
function prkey(){
var i;
for(i=1;i<=9;i++)
{
int x=1+(i*phi);
if(x%e==0)
{
d=x/e;
break;
}
}
}
encmsg=Math.pow(Integer.parseInt(msg),evalue)%n;
decmsg=pow(encmsg,d)%N;





Coding

Simple RSA:

```
import java.math.BigInteger;
import java.util.Scanner;

public class SimpleRSAFull {
// Convert string to BigInteger
   static BigInteger textToBigInt(String text) {
      return new BigInteger(1, text.getBytes());
   } // Convert BigInteger back to string

   static String bigIntToText(BigInteger val) {
      byte[] bytes = val.toByteArray();
      if (bytes[0] == 0) { // Remove leading zero if present
```

```
        byte[] tmp = new byte[bytes.length - 1];
        System.arraycopy(bytes, 1, tmp, 0, tmp.length);
        bytes = tmp;
    }
    return new String(bytes);
}public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    //  Choose two prime numbers
    BigInteger p = BigInteger.valueOf(61);
    BigInteger q = BigInteger.valueOf(53);

    // Calculate n and phi
    BigInteger n = p.multiply(q); // modulus
    BigInteger phi = p.subtract(BigInteger.ONE).multiply(q.subtract(BigInteger.ONE));

    // Choose public exponent e
    BigInteger e = BigInteger.valueOf(17);
    if (!e.gcd(phi).equals(BigInteger.ONE)) {
        System.out.println("e and phi(n) are not coprime!");
        sc.close();
        return;
    }

    //  Calculate private exponent d
    BigInteger d = e.modInverse(phi);

    //  Display keys
    System.out.println("Public Key (e, n): (" + e + ", " + n + ")");
    System.out.println("Private Key (d, n): (" + d + ", " + n + ")");

    // Take input message
    System.out.print("Enter a message: ");
    String message = sc.nextLine();
```

```
        // Encrypt entire message
        BigInteger plainInt = textToBigInt(message);
        if (plainInt.compareTo(n) >= 0) {
            System.out.println("Message too large for the chosen modulus. Use bigger primes!");
            sc.close();
            return;
        }
        BigInteger cipherInt = plainInt.modPow(e, n);

        // Decrypt the message
        BigInteger decryptedInt = cipherInt.modPow(d, n);
        String decryptedMessage = bigIntToText(decryptedInt);

        // Output results
        System.out.println("\nEncrypted (as number): " + cipherInt);
        System.out.println("Decrypted message: " + decryptedMessage);

        sc.close();
    }
}
```

Big integers:

```
import java.math.BigInteger;

import java.security.SecureRandom;

import java.nio.charset.StandardCharsets;

import java.util.ArrayList;

import java.util.List;

import java.util.Scanner;
```

```
public class RSAChunked {

    // Modular exponentiation

    static BigInteger power(BigInteger base, BigInteger expo, BigInteger mod) {

        return base.modPow(expo, mod);

    }    // Modular inverse

    static BigInteger modInverse(BigInteger e, BigInteger phi) {

        return e.modInverse(phi);

    }

    // Key generation

    static void generateKeys(BigInteger[] keys, int primeDigits) {

        SecureRandom random = new SecureRandom();

        int bitLength = (int) Math.round(primeDigits / 0.301);

        BigInteger p = BigInteger.probablePrime(bitLength, random);

        BigInteger q = BigInteger.probablePrime(bitLength, random);

        BigInteger n = p.multiply(q);

        BigInteger phi = p.subtract(BigInteger.ONE).multiply(q.subtract(BigInteger.ONE));

        BigInteger e = BigInteger.valueOf(65537); // Common choice

        if (!e.gcd(phi).equals(BigInteger.ONE)) {

            e = BigInteger.probablePrime(bitLength / 2, random);
```

```
    }
    BigInteger d = modInverse(e, phi);
    keys[0] = e; // Public exponent
    keys[1] = d; // Private exponent
    keys[2] = n; // Modulus
    // Show primes (for learning)
    System.out.println("Generated prime p (" + p.toString().length() + " digits): " + p);
    System.out.println("Generated prime q (" + q.toString().length() + " digits): " + q);
}// Split message into safe chunks
static List<BigInteger> encryptMessage(String message, BigInteger e, BigInteger n) {
    byte[] messageBytes = message.getBytes(StandardCharsets.UTF_8);
    // Max block size in bytes (n.bitLength() / 8) - 1 ensures < n
    int blockSize = (n.bitLength() / 8) - 1;
    List<BigInteger> encryptedBlocks = new ArrayList<>();
    for (int i = 0; i < messageBytes.length; i += blockSize) {
        int end = Math.min(i + blockSize, messageBytes.length);
        byte[] block = new byte[end - i];
        System.arraycopy(messageBytes, i, block, 0, block.length);
        BigInteger M = new BigInteger(1, block);
```

```
        BigInteger C = power(M, e, n);

        encryptedBlocks.add(C);

    }return encryptedBlocks;

  }// Decrypt chunks back to text

  static String decryptMessage(List<BigInteger> encryptedBlocks, BigInteger d, BigInteger
n) {StringBuilder result = new StringBuilder();

    for (BigInteger C : encryptedBlocks) {

        BigInteger M = power(C, d, n);

        byte[] blockBytes = M.toByteArray();

        // Remove leading zero if present

        if (blockBytes.length > 1 && blockBytes[0] == 0) {

            byte[] tmp = new byte[blockBytes.length - 1];

            System.arraycopy(blockBytes, 1, tmp, 0, tmp.length);

            blockBytes = tmp;

        }result.append(new String(blockBytes, StandardCharsets.UTF_8));

    }

    return result.toString();

  }
```

```
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    BigInteger[] keys = new BigInteger[3];
    int primeDigits = 30; // Each prime ~30 decimal digits
    generateKeys(keys, primeDigits);
    System.out.println("\nPublic Key (e, n): (" + keys[0] + ", " + keys[2] + ")");
    System.out.println("Private Key (d, n): (" + keys[1] + ", " + keys[2] + ")");
    System.out.print("\nEnter text message to encrypt: ");
    String message = sc.nextLine();
    // Encrypt in chunks
    List<BigInteger> encrypted = encryptMessage(message, keys[0], keys[2]);
    System.out.println("\nEncrypted Blocks:");
    for (BigInteger block : encrypted) {
        System.out.println(block);
    }// Decrypt chunks
    String decryptedMessage = decryptMessage(encrypted, keys[1], keys[2]);
    System.out.println("\nDecrypted Text: " + decryptedMessage)
    sc.close();
}
```

SCREENSHOTS:

```
D:\SEM V\Info Sec>javac SimpleRSAFull.java

D:\SEM V\Info Sec>java SimpleRSAFull
Public Key (e, n): (17, 999962000357)
Private Key (d, n): (529390588445, 999962000357)
Enter a message: Tamil

Encrypted (as number): 50889663380
Decrypted message: Tamil
```

```
D:\SEM V\Info Sec>java RSAChunked
Generated prime p (31 digits): 1188571743819791656822495429261
Generated prime q (30 digits): 684129506142613429411796691341

Public Key (e, n): (65537, 813137000114498911692995285479026061801198293077775316729001)
Private Key (d, n): (784798734581112953923997601237261112271653014680698604293073, 813137000114498911692995285479026061801
1982930777753167290 01)

Enter text message to encrypt: TransferThisMessage

Encrypted Blocks:
445684279533400020825561862406000797834238472878131728004474

Decrypted Text: TransferThisMessage
```

RESULT:

Thus, the implementation of RSA in virtual lab environment and in java have been done successfully

Evaluation

Parameter	Max Marks	Marks Obtained
Uniqueness of the Code	40	
Completion of experiment on time	5	
Documentation	20	
Simulation in Vlabs	10	
Total	75	
Signature of the faculty with Date		